

SET-A

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

You are building a text filtering tool for a data cleaning application. Users often input strings where words are separated by a special character. Some characters in each word appear only once (considered noise), while others appear multiple times (considered significant).

Your task is to develop a program that removes all unique (single-occurrence) characters from each word while keeping all occurrences of characters that appear more than once.

The program should:

- Read a string containing multiple words separated by a user-specified character
- Read the separator character
- Split the string manually (no built-in split)
- For each word, remove characters that appear only once
- Print the filtered words

Input	Output
heellllloo-worrlld -	The words are: eelllloo rr

SET-B

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

You are developing a text analyzer where users input strings with repeated characters and special separators. In this case, characters that appear only once are considered meaningful, while repeated characters are treated as noise.

Your task is to develop a program that keeps only the characters that appear exactly once in each word and removes all repeated characters.

The program should:

- Read a string containing multiple words separated by a user-specified character
- Read the separator character
- Split the string manually (no built-in split)
- For each word, keep only characters that appear once
- Print the cleaned words

Input	Output
heellllloo#worrlld #	The words are: h wold

Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.Scanner;

public class Question1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String text = sc.nextLine();
        char separator = sc.nextLine().charAt(0);

        // add separator at end
        text = text + separator;

        System.out.println("The words are:");

        String word = "";

        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);

            if (ch == separator) {
                String result = "";

                for (int j = 0; j < word.length(); j++) {
                    char current = word.charAt(j);
                    int count = 0;

                    for (int k = 0; k < word.length(); k++) {
                        if (word.charAt(k) == current) {
                            Count++;
                        }
                    }

                    // keep only repeated characters
                    if (count > 1) {
                        result = result + current;
                    }
                }

                System.out.println(result);
                word = "";
            } else {
                word = word + ch;
            }
        }
    }
}
```

RUBRIC		
SL	Points to Meet	Marks [10]
1	Correct input handling (string + separator)	2
2	Proper manual splitting using loop (no built-in)	2
3	Correct logic for counting character frequency	2
4	Correct filtering condition (Set A: >1, Set B: ==1)	2
5	Correct output formatting and handling all words	2

SET - [B]**SOLUTION**

```
import java.util.Scanner;

public class Question2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String text = sc.nextLine();
        char separator = sc.nextLine().charAt(0);

        // add separator at end
        text = text + separator;

        System.out.println("The words are:");

        String word = "";

        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);

            if (ch == separator) {
                String result = "";

                for (int j = 0; j < word.length(); j++) {
                    char current = word.charAt(j);
                    int count = 0;

                    for (int k = 0; k < word.length(); k++) {
                        if (word.charAt(k) == current) {
                            count++;
                        }
                    }

                    // keep only unique characters
                    if (count == 1) {
                        result = result + current;
                    }
                }

                System.out.println(result);
                word = "";
            } else {
                word = word + ch;
            }
        }
    }
}
```

RUBRIC		
SL	Points to Meet	Marks [10]
1	Correct input handling (string + separator)	2
2	Proper manual splitting using loop (no built-in)	2
3	Correct logic for counting character frequency	2
4	Correct filtering condition (Set A: >1, Set B: ==1)	2
5	Correct output formatting and handling all words	2

SET-A

Name:	ID:	Section:
-------	-----	----------

Question 1 [C03] [10 Points]

You are given an integer array and you need to take a number as input from the user. Your task is to update the array by replacing the duplicate values of the array with zero. The updated array should be sorted in ascending order. Finally, you have to print the position of the number (you took as input from the user) from the updated array. Print **"Not found"** if the number is not in the array.

Can't use built-in sort for the problem. Can't create a new array for this problem.

Input	Output
<pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=2</pre>	Updated array: 6, 2, 0, 1, 0, 9, 0 Sorted array: 0,0,0,1,2,6,9 Output: Position of 2 = 5
<pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=5</pre>	Updated array: 6, 2, 0, 1, 0, 9, 0 Sorted array: 0,0,0,1,2,6,9 Output: Position of 5 = Not found

SET-B

Name:	ID:	Section:
-------	-----	----------

Question 1 [C03] [10 Points]

You are given an integer array and you need to take a number as input from the user. Your task is to create a new array that will contain only the unique (distinct) elements of the given array. The new array should be sorted in descending order. Finally, you have to print the position of the number (you took as input from the user) in the new array. Print **"Not found"** if the number is not in the array.

Can't use built-in sort for the problem.

Input	Output
<pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=6</pre>	<pre>New array: 6, 2, 1, 9 Sorted array: 9,6,2,1 Output: Position of 6 = 2</pre>
<pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=7</pre>	<pre>New array: 6, 2, 1, 9 Sorted array: 9,6,2,1 Output: Position of 7 = Not found</pre>

Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int[] arr = {6, 2, 6, 1, 2, 9, 1};
        int elem = 5;

        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < i; j++) {
                if (arr[i] == arr[j]) {
                    arr[i] = 0;
                    break;
                }
            }
        }

        System.out.println("Updated Array:");
        for (int num : arr) {
            System.out.print(num + " ");
        }

        for (int i = 0; i < arr.length - 1; i++) {
            int minIndex = i;

            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }

            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }

        int pos = -1;
        System.out.println("\nSorted Array:");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
            if (arr[i] == elem){
                pos = i+1;
            }
        }
        if(pos == -1){
            System.out.printf("\nOutput: \nPosition of %s = Not found", elem);
        }else{
            System.out.printf("\nOutput: \nPosition of %s = %s", elem,pos);
        }
    }
}
```

```
}
```

RUBRIC

SL	Points to Meet	Marks [10]
1	Handle Duplicates by replacing with 0 in the array	3
2	Sort the array in ascending order	3
3	Search and find the position	2
4	Follow the constraint (No built-in sort + No new array)	1
5	Maintain Output format (updated array, sorted array, output position)	1

SET - [B]

SOLUTION

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] arr = {6, 2, 6, 1, 2, 9, 1};
        int elem = 6;

        int[] newArr = new int[arr.length];
        int count = 0;

        for (int i = 0; i < arr.length; i++) {
            boolean isDuplicate = false;

            for (int j = 0; j < count; j++) {
                if (arr[i] == newArr[j]) {
                    isDuplicate = true;
                    break;
                }
            }

            if (!isDuplicate) {
                newArr[count] = arr[i];
                count++;
            }
        }

        System.out.println("\nNew array:");
        for (int i = 0; i < count; i++) {
            System.out.print(newArr[i] + " ");
        }

        for (int i = 0; i < newArr.length - 1; i++) {
            int maxIndex = i;

            for (int j = i + 1; j < newArr.length; j++) {
                if (newArr[j] > newArr[maxIndex]) {
                    maxIndex = j;
                }
            }

            int temp = newArr[i];
            newArr[i] = newArr[maxIndex];
            newArr[maxIndex] = temp;
        }

        int pos = -1;
        System.out.println("\nSorted Array:");
        for (int i = 0; i < count; i++) {
            System.out.print(newArr[i] + " ");
            if (newArr[i] == elem){
                pos = i+1;
            }
        }
        if(pos == -1){
            System.out.printf("\nOutput: \nPosition of %s = Not found", elem);
        }
    }
}
```

```

    }else{
        System.out.printf("\nOutput: \nPosition of %s = %s", elem,pos);
    }
}
}

```

RUBRIC

SL	Points to Meet	Marks [10]
1	Create unique elements in new array	3
2	Sort the array in descending order	3
3	Search and find the position	2
4	Follow the constraint (No built-in sort)	1
5	Maintain Output Format (new array, sorted array, output position)	1